FFF Audit Report

Version 1.0

Published By : Dionis Xhaferi

Security Audit Findings for FFF Token Contract

High Severity

[H-1] Missing Access Control on `_mint` Function

Description: The `_mint` function is marked as `public` instead of `internal`, which means anyone can call it directly. While it has the `onlyOwner` modifier, this is insufficient as the function should be internal to prevent direct calls.

Impact: High - Malicious actors could potentially mint tokens by directly calling the `_mint` function, bypassing intended controls.

Proof of Concept:

```
function _mint(address account, uint256 amount) public virtual onlyOwner {
// Anyone can call this function directly
  }
```

Recommended Mitigation: Change the visibility from `public` to `internal`:

```
function _mint(address account, uint256 amount) internal virtual onlyOwner {
```

Medium Severity

[M-1] Missing Zero Address Checks in Transfer Functions

Description: While the `_transfer` function checks for zero addresses, the `transfer` and `transferFrom` functions don't have explicit checks for zero addresses in their parameters.

Impact: Medium - Could lead to accidental token burns or transfers to invalid addresses.

Proof of Concept:

```
function transfer(address to, uint256 amount) public virtual override returns (bool) {
// No explicit zero address check for 'to' address
     owner = _msgSender();
_transfer(owner, to, amount); return
     true;
 }
```

Recommended Mitigation: Add explicit zero address checks in the public functions:

```
 function transfer(address to, uint256 amount) public virtual override returns (bool) {
     require(to != address(0), "BEP20: transfer to the zero address");
address owner = _msgSender();
_transfer(owner, to, amount); return
     true;
 }
```

[M-2] Potential Integer Overflow in `_mint` Function

Description: The `_mint` function multiplies the input amount by `10**_decimals` without checking for overflow.

Impact: Medium - Could lead to unexpected behavior if the input amount is too large.

Proof of Concept:

```
 function _mint(address account, uint256 amount) public virtual onlyOwner {
     uint256 Amount = amount * 10**_decimals; // Potential overflow
     // ...
 }
```

Recommended Mitigation: Add overflow check:

```
 function _mint(address account, uint256 amount) internal virtual onlyOwner {
     require(amount <= type(uint256).max / 10**_decimals, "BEP20: amount too large");
     uint256 Amount = amount * 10**_decimals;
     // ...
 }
```

Low Severity

[L-1] Missing Event Emission for Ownership Transfer in Constructor

Description: The constructor sets an initial owner but doesn't emit an ownership transfer event.

Impact: Low - Missing event emission makes it harder to track ownership changes.

Proof of Concept:

```
constructor() {
_owner = address(0xcb98eafa8aCF87545c6773748d256eBFf7B0a128);
// Missing OwnershipTransferred event
 }
```

Recommended Mitigation: Add event emission in constructor:

```
constructor() {
_owner = address(0xcb98eafa8aCF87545c6773748d256eBFf7B0a128); emit
     OwnershipTransferred(address(0), _owner);
 }
```

Informational

[I-1] Hardcoded Owner Address

Description: The owner address is hardcoded in the constructor.

Impact: Informational - Makes the contract less flexible and harder to deploy to different networks.

Proof of Concept:

```
constructor() {
_owner = address(0xcb98eafa8aCF87545c6773748d256eBFf7B0a128);
     // ...
 }
```

Recommended Mitigation: Consider making the owner address a constructor parameter:

```
constructor(address initialOwner) {
require(initialOwner != address(0), "Ownable: initial owner is the zero address");
_owner = initialOwner;
emit OwnershipTransferred(address(0), _owner);
  }
```

[I-2] Missing Pausable Functionality

Description: The contract doesn't implement pausable functionality, which could be useful for emergency situations.

Impact: Informational - Lack of emergency stop mechanism.

Recommended Mitigation: Consider implementing OpenZeppelin's Pausable contract and adding pausable modifiers to critical functions.

[I-4] Missing Blacklist Functionality

Description: The contract doesn't implement blacklist functionality, which could be useful for regulatory compliance.

Impact: Informational - Lack of ability to block malicious addresses.

Recommended Mitigation: Consider implementing a blacklist mechanism with appropriate access controls.